Proyecto final hacking ético

Integrantes

Camilo Alvarez Gutirrez

Maria Fernanda Mercado Fernandez

Albert David Peña Artunduag

Ronni Enrique Perez Machado

Hacking ético Grupo 1

Docente

Sergio Luis Lubo Argumedo

10/09/2026

4.1. Caracterización del sistema

- Aplicación o API seleccionada.
  DVWA
- Propósito y principales funcionalidades.
  DVWA es una aplicación web deliberadamente vulnerable que utilizaremos para estudiar fallos de seguridad en un entorno controlado.
- Tecnologías utilizadas.
  PHP 8.4.27
  Apache 2.4.69
  MariaDB/MySQL 11.8.9-MariaDB
- Arquitectura o componentes principales.
  Backend: PHP.
  Servidor web: Apache.
  Base de datos: MariaDB/MySQL.
  Interfaz: HTML, CSS y JavaScript.
- Entorno utilizado para las pruebas.
  Kali Linux sobre VirtualBox.
- Superficie de ataque identificada.
  http://127.0.0.1/DVWA/

4.2. Análisis de vulnerabilidades

4.2.1. Vulnerabilidad 1: Inyección SQL (SQL Injection)

Descripción técnica

Durante el análisis del módulo SQL Injection de DVWA se identificó una vulnerabilidad relacionada con la construcción insegura de consultas SQL. La aplicación recibe un parámetro denominado id mediante la variable $_REQUEST y posteriormente lo incorpora directamente en una consulta SQL.

Causa de la vulnerabilidad

La causa principal es la concatenación directa de datos proporcionados por el usuario dentro de una instrucción SQL, sin utilizar consultas preparadas ni mecanismos adecuados para separar los datos de las instrucciones.

Análisis de explotación

Inicialmente se realizó una consulta legítima utilizando el identificador 1, obteniendo el registro correspondiente.



Posteriormente, se inspeccionó el archivo vulnerabilities/sqli/source/low.php, identificando que el parámetro recibido se incorpora directamente en la consulta SQL.



Para demostrar la vulnerabilidad se introdujo una entrada especialmente construida que modificó la condición lógica de la consulta. Como resultado, la aplicación devolvió múltiples registros de usuarios en lugar de limitarse al identificador solicitado.



Impacto

Esta vulnerabilidad puede comprometer la confidencialidad de la información almacenada en la base de datos. Dependiendo de los permisos de conexión y las operaciones disponibles, también podría afectar la integridad y disponibilidad de la información.

Clasificación OWASP

OWASP Top 10 2021: A03 – Injection.

Mitigación

- **Código fuente**

  sudo nano vulnerabilities/sqli/source/low.php

  case MYSQL:

  ```
  // Check database
  $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
  $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( '<pre>' .
((is_object($GLOBALS["___mysqli_ston"])) ? mysqli_error($GLOBALS["___mysqli_ston"]) :
(($___mysqli_res = mysqli_connect_error()) ? $___mysqli_res : false)) . '</pre>' );
  // Get results
  while( $row = mysqli_fetch_assoc( $result ) ) {
      // Get values
      $first = $row["first_name"];
      $last  = $row["last_name"];
      // Feedback for end user
      $html .= "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
  }
  mysqli_close($GLOBALS["___mysqli_ston"]);
  break;
  ```

- **Configuración**

  sudo nano vulnerabilities/sqli/source/low.php

  case MYSQL:

  ```
  case MYSQL:
    // Consulta preparada para prevenir SQL Injection
    $query = "SELECT first_name, last_name FROM users WHERE user_id = ?";
    $stmt = mysqli_prepare($GLOBALS["___mysqli_ston"], $query);
    if (!$stmt) {
       die('<pre>Error al preparar la consulta.</pre>'); }
    // Vincular el parámetro como una cadena
    mysqli_stmt_bind_param($stmt, "s", $id);
    // Ejecutar la consulta
    mysqli_stmt_execute($stmt);
    // Obtener los resultados
    $result = mysqli_stmt_get_result($stmt);
    // Mostrar los resultados
    while ($row = mysqli_fetch_assoc($result)) {
       $first = $row["first_name"];
       $last = $row["last_name"];
       $html .= "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
    }
    // Cerrar la consulta preparada
    mysqli_stmt_close($stmt);
  ```

```
mysqli_close($GLOBALS["___mysqli_ston"]);
break;
```

- **Justificación**
  Se seleccionaron las consultas preparadas porque constituyen uno de los mecanismos más efectivos para prevenir ataques de inyección SQL. Al separar la estructura de la consulta de los datos recibidos, se evita que caracteres especiales o fragmentos de código SQL introducidos por el usuario alteren la lógica de la instrucción.
  Esta solución es preferible a depender exclusivamente de filtros o validaciones de caracteres, ya que estos pueden resultar insuficientes frente a diferentes formas de manipulación de entradas.

4.2.2. Vulnerabilidad 2: Stored Cross-Site Scripting (Stored XSS)

Descripción técnica

Durante la evaluación de seguridad de DVWA se seleccionó el módulo XSS (Stored), con el objetivo de identificar posibles fallos en el tratamiento y visualización de información proporcionada por los usuarios.

Stored XSS es una vulnerabilidad que permite almacenar contenido que posteriormente puede ser interpretado y ejecutado por el navegador cuando otro usuario consulta la información.

Componente afectado

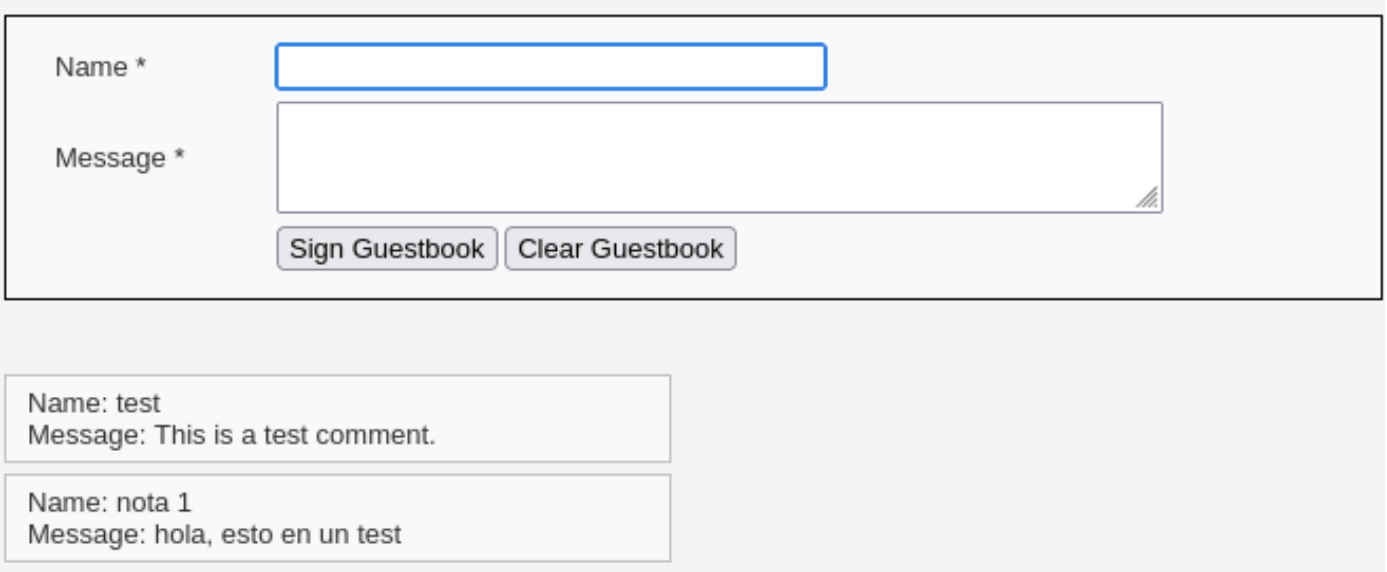
Módulo XSS Stored de DVWA.

Causa de la vulnerabilidad

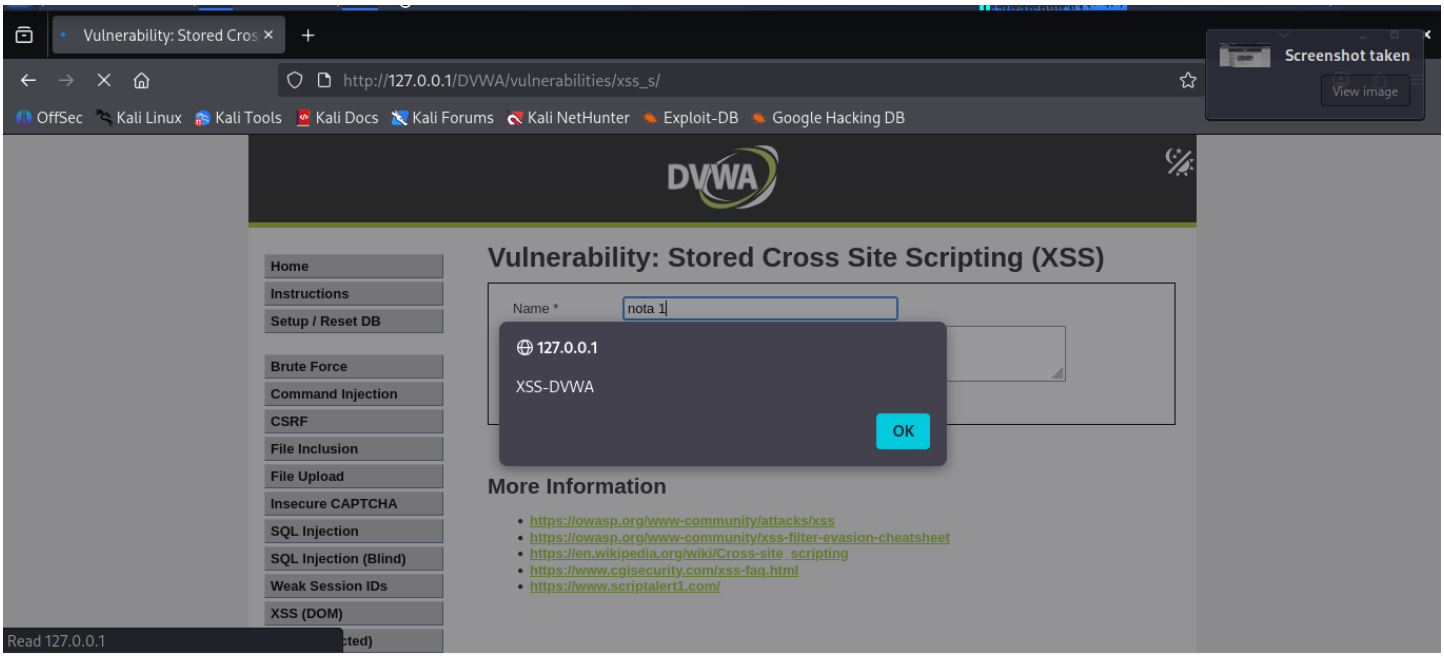
La causa que se investigará es la ausencia de una codificación de salida adecuada al mostrar contenido almacenado en la aplicación web. Esto puede permitir que el navegador interprete contenido introducido por un usuario como código ejecutable.

Análisis de explotación

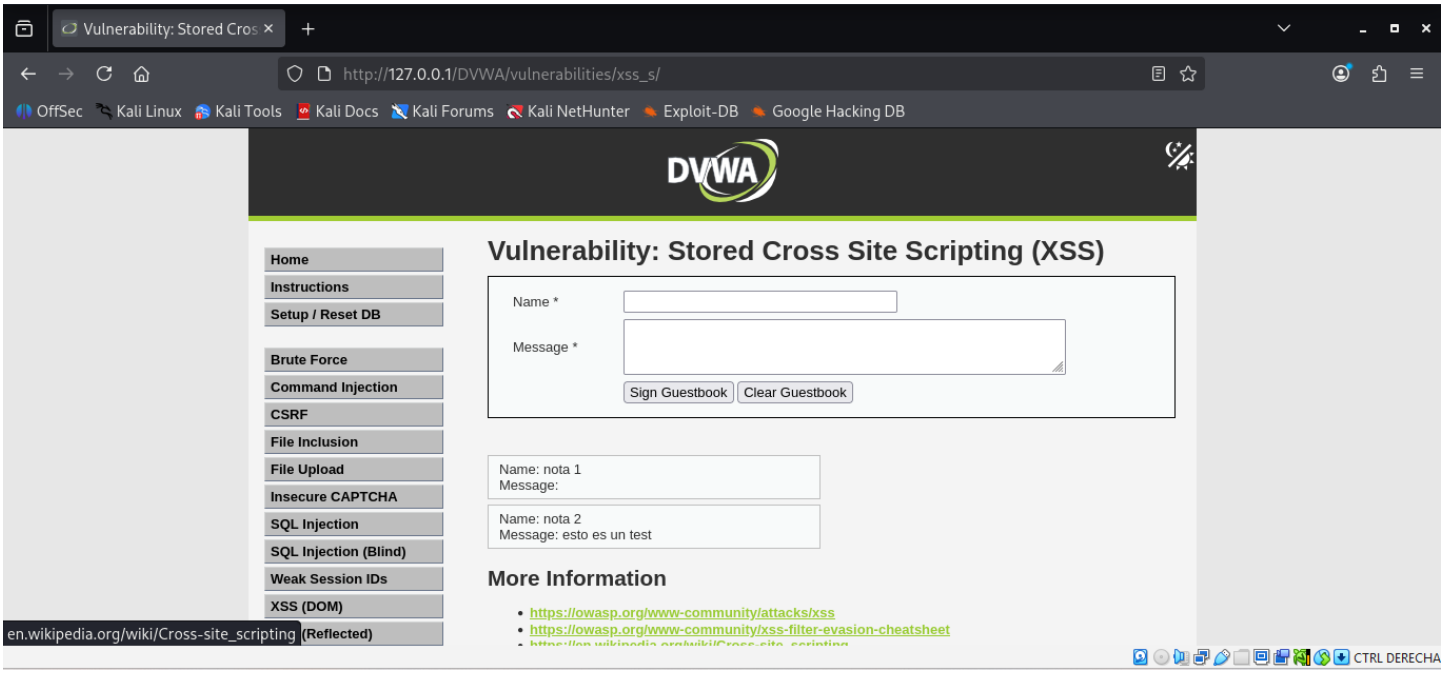
Se realizo inicialmente el registro de un comentario legítimo para comprobar el funcionamiento normal del libro de visitas.



Posteriormente se introduce una prueba de concepto JavaScript inocua en el campo de comentarios, con el propósito de comprobar si el contenido almacenado es interpretado por el navegador.



Se verifica también que el contenido permanece almacenado después de actualizar la página.



Impacto

La vulnerabilidad puede permitir la ejecución de JavaScript en el navegador de los usuarios que visualicen el contenido afectado. El impacto dependerá del contexto de la aplicación y de los privilegios disponibles.

Clasificación

OWASP Top 10 2021: A03 – Injection.

Mitigación

sudo nano /var/www/html/DVWA/dvwa/includes/dvwaPage.inc.php

- **Código fuente**

```
// XSS Stored guestbook function --
function dvwaGuestbook() {
        $query  = "SELECT name, comment FROM guestbook";
        $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query );

        $guestbook = '';

        while( $row = mysqli_fetch_row( $result ) ) {
                if( dvwaSecurityLevelGet() == 'impossible' ) {
                        $name    = htmlspecialchars( $row[0] );
                        $comment = htmlspecialchars( $row[1] );
                }
                else {
                        $name    = $row[0];
                        $comment = $row[1];
                }

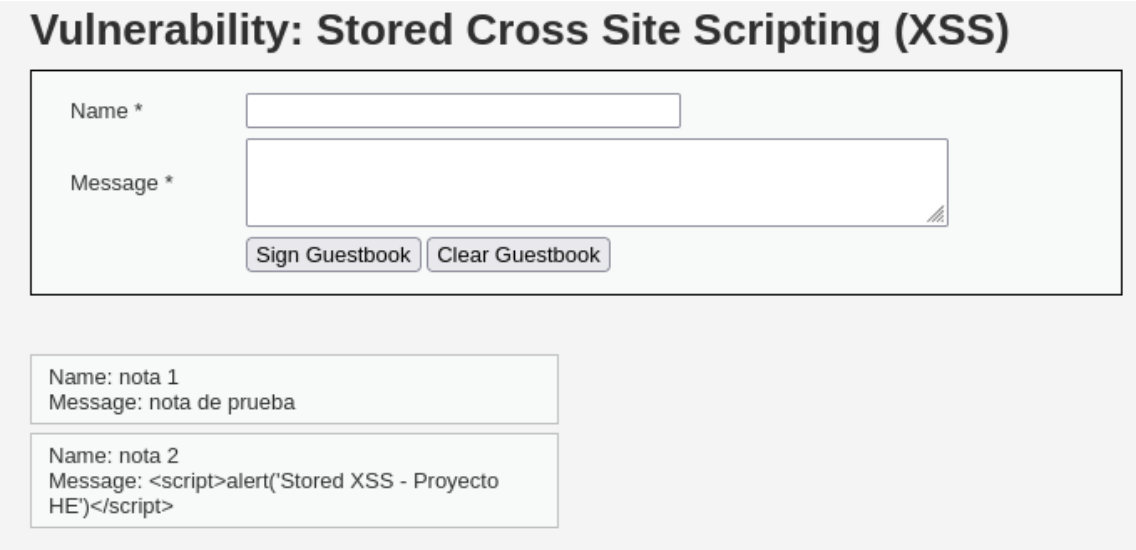
                $guestbook .= "<div id=\"guestbook_comments\">Name: {$name}<br />" . "Message: {$comment}<br /></div>\n";
        }
        return $guestbook;
}
// -- END (XSS Stored guestbook)
```

- **Configuración**

```
$name = htmlspecialchars(
  $row[0],
  ENT_QUOTES | ENT_SUBSTITUTE,
  'UTF-8'
);
```

```
$comment = htmlspecialchars(
  $row[1],
  ENT_QUOTES | ENT_SUBSTITUTE,
  'UTF-8'
);
```

- **Justificación**
  La solución utiliza codificación de salida contextual mediante htmlspecialchars(), aplicada a los datos recuperados de la base de datos antes de insertarlos en el documento HTML.
  Esta medida impide que el navegador interprete los caracteres especiales como elementos HTML ejecutables. Se aplica tanto al nombre como al comentario, porque ambos campos son controlados por el usuario.



4.2.3. Vulnerabilidad 3: Brute Force (fuerza bruta)

Descripción técnica

El módulo de autenticación de DVWA (Brute Force) no implementa ningún mecanismo de control sobre el número de intentos de inicio de sesión que un usuario puede realizar. Al enviar repetidamente combinaciones de usuario y contraseña mediante el formulario de login, el sistema procesa cada solicitud de manera independiente, sin verificar cuántos intentos previos ha realizado el mismo cliente, sin aplicar retardos progresivos y sin bloquear la cuenta tras múltiples fallos consecutivos. Esto permite automatizar un ataque de fuerza bruta o de diccionario contra las credenciales de cualquier usuario del sistema.

Componente afectado

Formulario de autenticación del módulo Brute Force de DVWA, accesible en la ruta /DVWA/vulnerabilities/brute/, el cual recibe las credenciales (username, password) mediante una solicitud HTTP GET.

Posibles causas de la vulnerabilidad

Ausencia de controles de autenticación robustos en el servidor, específicamente:

- No existe un contador de intentos fallidos asociado al usuario o a la sesión/IP del cliente.
- No hay bloqueo temporal ni permanente de la cuenta tras múltiples fallos.
- No se implementa retardo progresivo entre intentos.
- No se exige CAPTCHA ni ningún otro mecanismo anti-automatización.
- Las credenciales viajan como parámetros de una petición GET, lo cual además facilita su automatización y registro en logs/historial.

Análisis de explotación

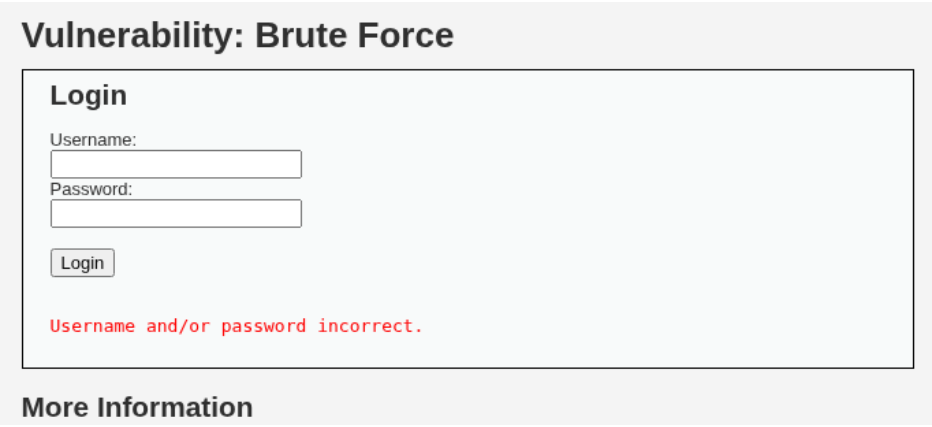
Mediante pruebas manuales y repetidas sobre el formulario de login, se evidenció que:

Enviar múltiples intentos fallidos consecutivos (se probaron más de 10 intentos seguidos) no genera ningún bloqueo, advertencia ni CAPTCHA.

El tiempo de respuesta del servidor se mantiene constante independientemente del número de intentos previos fallidos, lo que descarta la existencia de un retardo progresivo.

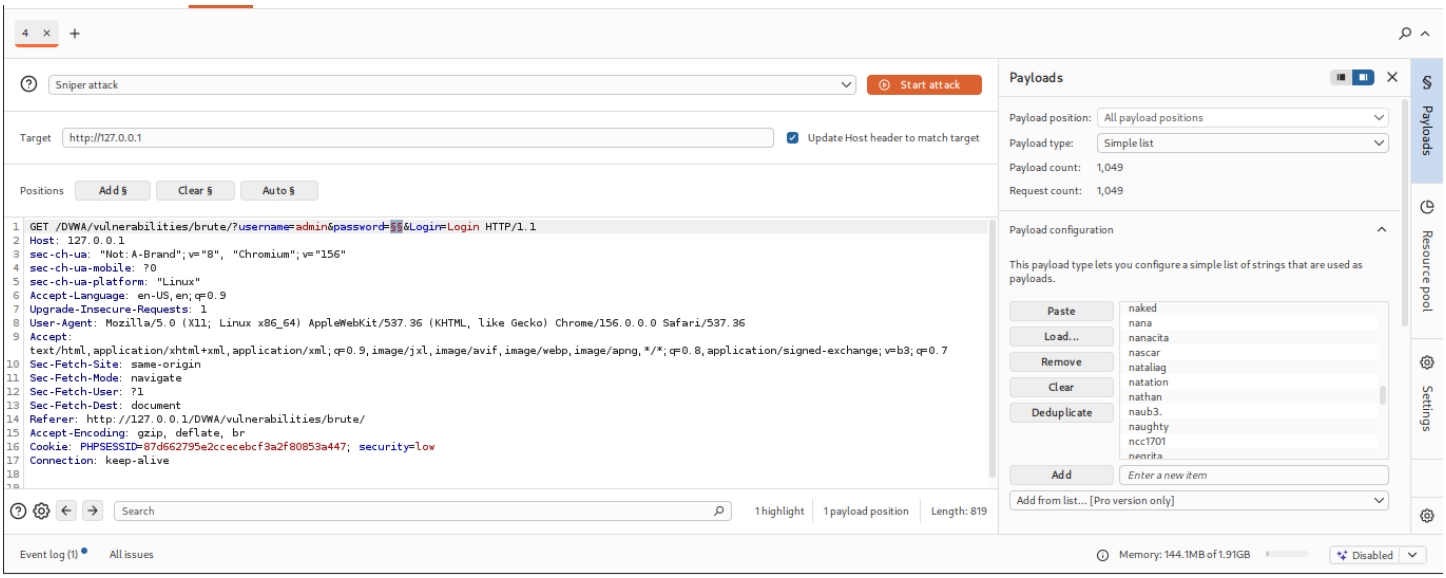
Cada solicitud es determinística y autocontenida (no depende de un token dinámico ni de un estado previo que el atacante no pueda replicar), por lo que el proceso es completamente automatizable.

Esta ausencia de restricciones se confirmará formalmente en la fase de explotación controlada

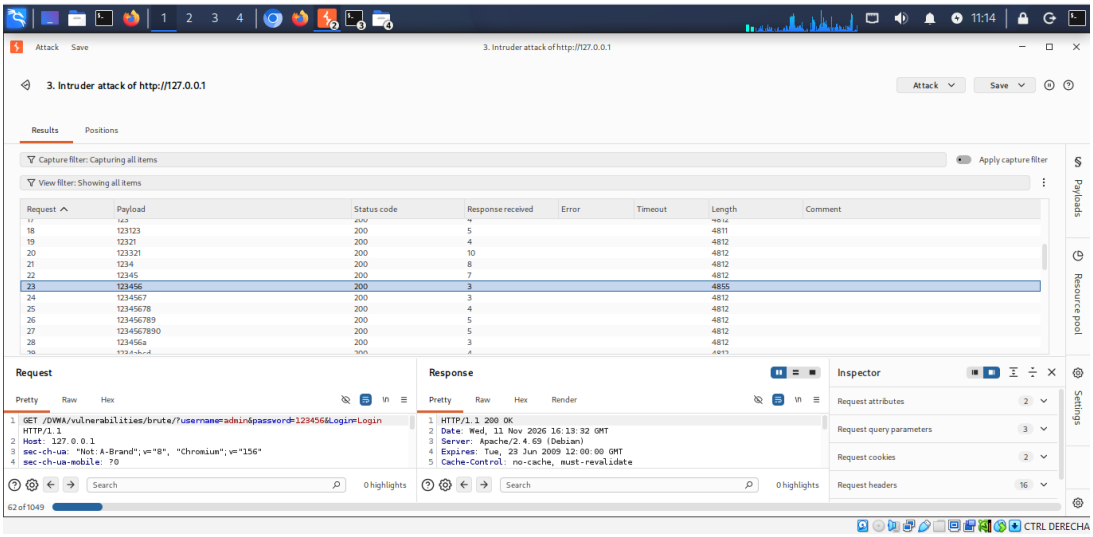


mediante Burp Suite Intruder, donde se automatizará el envío de un diccionario de contraseñas contra un usuario conocido (admin) para demostrar que el sistema permite un número ilimitado de intentos sin interrupción.

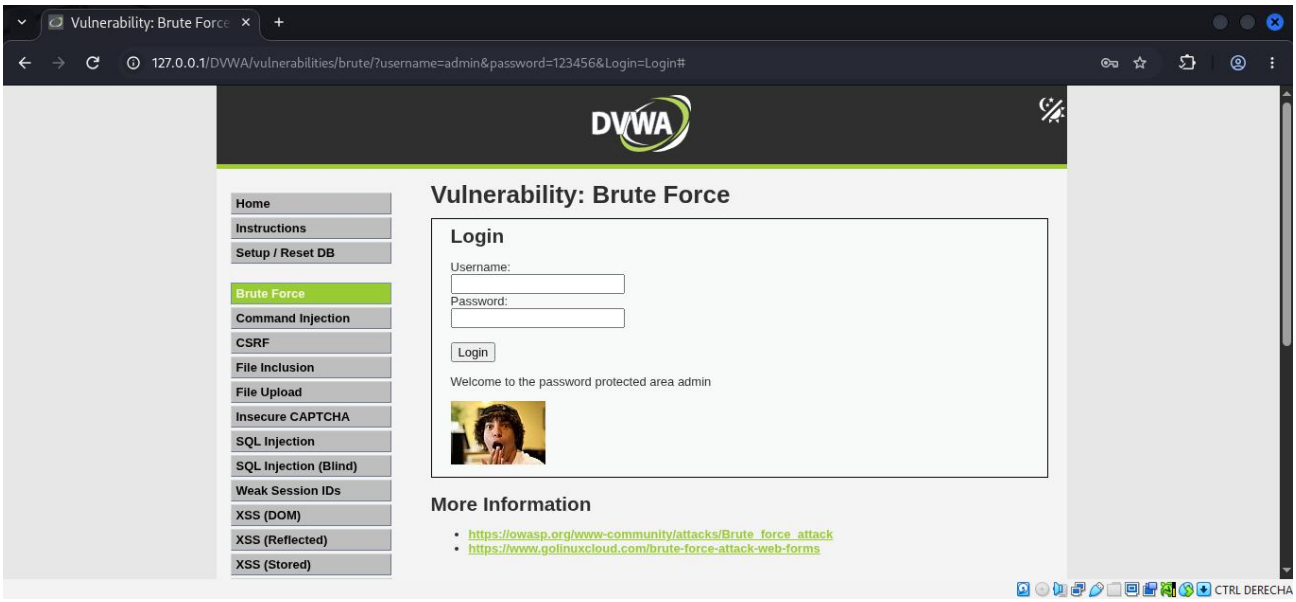
El directorio de contraseñas a usar va a ser best1050 un directorio con las 1050 contraseñas más usadas.



Encontramos una contraseña con un tiempo de respuesta anormal el cual fue 123456



Al introducirla nos sale que esa es la contraseña de admi



Clasificación OWASP

OWASP Web Application Security Top 10 (2021) — A07:2021: Identification and Authentication Failures.

Migración

- **Código original**

```
 sudo  cat /var/www/html/DVWA/vulnerabilities/brute/source/low.php
<?php

if( isset( $_GET[ 'Login' ] ) ) {
    // Get username
    $user = $_GET[ 'username' ];

    // Get password
    $pass = $_GET[ 'password' ];
    $pass = md5( $pass );

    // Check the database
    $query  = "SELECT * FROM `users` WHERE user = '$user' AND password = '$pass';";
    $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( '<pre>' .
((is_object($GLOBALS["___mysqli_ston"])) ? mysqli_error($GLOBALS["___mysqli_ston"]) :
(($___mysqli_res = mysqli_connect_error()) ? $___mysqli_res : false)) . '</pre>' );

    if( $result && mysqli_num_rows( $result ) == 1 ) {
        // Get users details
        $row    = mysqli_fetch_assoc( $result );
        $avatar = $row["avatar"];

        // Login successful
        $html .= "<p>Welcome to the password protected area {$user}</p>";
        $html .= "<img src=\"{$avatar}\" />";
    }
    else {
        // Login failed
        $html .= "<pre><br />Username and/or password incorrect.</pre>";
    }

    ((is_null($___mysqli_res = mysqli_close($GLOBALS["___mysqli_ston"]))) ? false :
$___mysqli_res);
}
?>
```

- **Código nuevo**

```
<?php
```

```
// --- Mitigación: control de intentos fallidos ---
if( !isset( $_SESSION[ 'login_attempts' ] ) ) {
    $_SESSION[ 'login_attempts' ] = 0;
    $_SESSION[ 'lockout_time' ]   = 0;
}

$max_intentos   = 5;
$tiempo_bloqueo = 30; // segundos

$bloqueado = ( $_SESSION[ 'login_attempts' ] >= $max_intentos &&
          ( time() - $_SESSION[ 'lockout_time' ] ) < $tiempo_bloqueo );

if( $bloqueado ) {
    $restante = $tiempo_bloqueo - ( time() - $_SESSION[ 'lockout_time' ] );
    $html .= "<pre><br />Demasiados intentos fallidos. Intenta de nuevo en {$restante}
segundos.</pre>";
}
elseif( isset( $_GET[ 'Login' ] ) ) {

    // Solo reinicia si realmente hubo un bloqueo previo y ya expiró
    if( $_SESSION[ 'lockout_time' ] > 0 &&
      ( time() - $_SESSION[ 'lockout_time' ] ) >= $tiempo_bloqueo ) {
        $_SESSION[ 'login_attempts' ] = 0;
        $_SESSION[ 'lockout_time' ]   = 0;
    }

    // Get username
    $user = $_GET[ 'username' ];

    // Get password
    $pass = $_GET[ 'password' ];
    $pass = md5( $pass );

    // Check the database
    $query  = "SELECT * FROM `users` WHERE user = '$user' AND password = '$pass';";
    $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( '<pre>' .
((is_object($GLOBALS["___mysqli_ston"])) ? mysqli_error($GLOBALS["___mysqli_ston"]) :
(($___mysqli_res = mysqli_connect_error()) ? $___mysqli_res : false)) . '</pre>' );

    if( $result && mysqli_num_rows( $result ) == 1 ) {
        $row    = mysqli_fetch_assoc( $result );
```

```
            $avatar = $row["avatar"];

            $_SESSION[ 'login_attempts' ] = 0;

            $html .= "<p>Welcome to the password protected area {$user}</p>";
            $html .= "<img src=\"{$avatar}\" />";
        }
        else {
            $_SESSION[ 'login_attempts' ]++;
            sleep( $_SESSION[ 'login_attempts' ] );

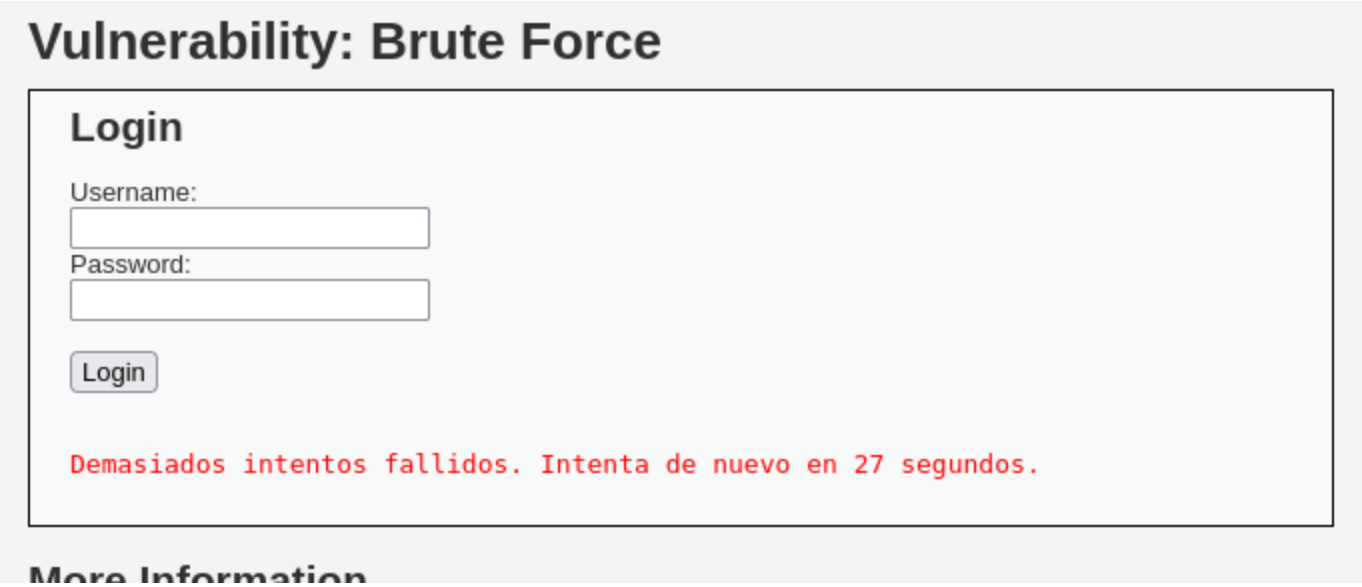
            if( $_SESSION[ 'login_attempts' ] >= $max_intentos ) {
                $_SESSION[ 'lockout_time' ] = time();
            }

            $html .= "<pre><br />Username and/or password incorrect.</pre>";
        }

        ((is_null($___mysqli_res = mysqli_close($GLOBALS["___mysqli_ston"]))) ? false :
    $___mysqli_res);
    }

    ?>
```

- **Justificación**
  La mitigación implementada ataca directamente la causa raíz de la vulnerabilidad: la ausencia total de control sobre los intentos de autenticación. Al incorporar un contador de intentos fallidos por sesión, un retardo progresivo (sleep) y un bloqueo temporal de 30 segundos tras cinco fallos consecutivos —verificando correctamente si el bloqueo ya expiró antes de reiniciar el contador—, se elimina la posibilidad de automatizar un ataque de fuerza bruta en un tiempo práctico, ya que cada intento fallido se vuelve más costoso para el atacante hasta llegar al bloqueo. Al mismo tiempo, el reinicio del contador únicamente tras un login exitoso o tras expirar el bloqueo garantiza que un usuario legítimo no se vea afectado por intentos previos fallidos, y al mantener toda la lógica dentro del mismo archivo low.php (sin depender de CAPTCHA ni de código de otros niveles de dificultad) se cumple la restricción de modificar un único archivo propio, conservando una solución autocontenida, verificada mediante pruebas funcionales y fácilmente justificable desde el punto de vista técnico.



4.2.4. Vulnerabilidad 4 Unrestricted File Upload

Descripción técnica

La vulnerabilidad File Upload consiste en la implementación incorrecta de los mecanismos de validación de archivos que permite a un usuario subir archivos potencialmente peligrosos al servidor.
En la aplicación DVWA, configurada con nivel de seguridad Low, se identificó que el formulario de subida permite cargar archivos con extensión `.php`, sin aplicar restricciones suficientes para impedir su almacenamiento.
Esta condición representa un riesgo importante, ya que, si el servidor permite interpretar y ejecutar los archivos PHP almacenados como una shell reverse.

Componente afectado

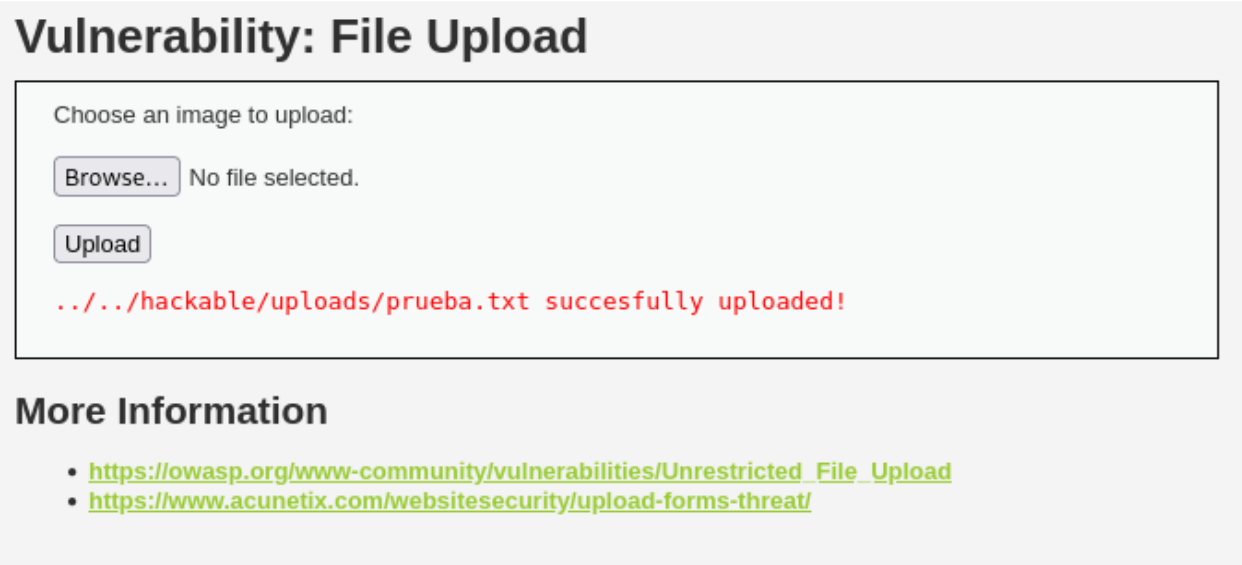
módulo File Upload de DVWA.

Análisis de explotación

Para comprobar la existencia de la vulnerabilidad se realizaron pruebas controladas sobre la instalación local de DVWA.

Inicialmente se creó un archivo de texto llamado `prueba.txt`, cuyo contenido era un mensaje de prueba. Al subirlo mediante el formulario, la aplicación mostró el siguiente resultado:

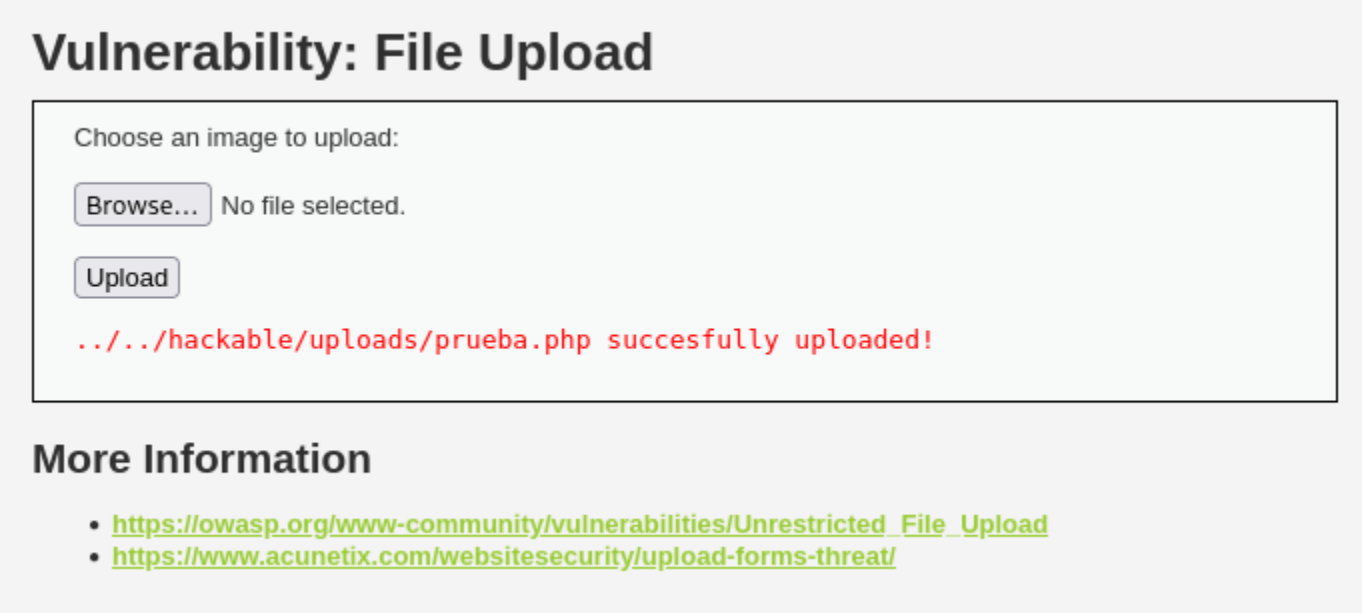
`../../hackable/uploads/prueba.txt successfully uploaded!`



Posteriormente se creó un archivo denominado `prueba.php`, que contenía únicamente una instrucción PHP para mostrar el mensaje «Ejecución PHP confirmada».

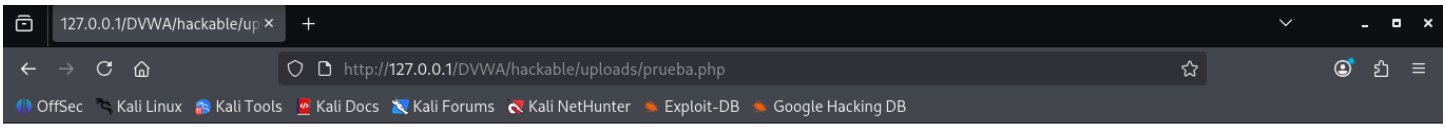
El archivo fue seleccionado y enviado mediante el mismo formulario. La aplicación respondió:

`../../hackable/uploads/prueba.php successfully uploaded!`



Finalmente, se accedió directamente a la ruta del archivo almacenado mediante el navegador. El servidor interpretó el código PHP y mostró el mensaje definido en el archivo.

Este resultado confirmó que la aplicación permite subir archivos PHP y que el servidor web ejecuta el código almacenado.



Ejecución PHP confirmada

Impacto

La explotación de esta vulnerabilidad puede comprometer la seguridad del servidor y de la aplicación web.

Entre los posibles impactos se encuentran:

- Ejecución de código no autorizado en el servidor.

- Modificación o eliminación de archivos accesibles para el proceso web.

- Acceso indebido a información almacenada en el servidor.

- Compromiso de otros componentes de la aplicación.

- Posibilidad de utilizar el servidor como punto de entrada para ataques adicionales.

El impacto real depende de los permisos del usuario con el que se ejecuta el servidor web y de las restricciones adicionales del entorno.

Clasificación OWASP

OWASP Top 10 – A04:2021 Insecure Design (Diseño inseguro)

Mitigación

Para corregir la vulnerabilidad identificada se modificó el archivo low.php, encargado de procesar las solicitudes de subida de archivos en el módulo File Upload de DVWA.

En la versión corregida se implementaron diferentes controles de seguridad:

Lista blanca de extensiones: se restringieron los archivos permitidos a los formatos JPG, JPEG y PNG, rechazando extensiones como PHP y TXT.

Validación del tipo MIME: se utilizó la función finfo_file() para identificar el tipo de contenido real del archivo y compararlo con los formatos permitidos.

Validación del contenido: se incorporó getimagesize() para comprobar que el archivo corresponda a una imagen válida.

Restricción del tamaño: se estableció un límite máximo de 5 MB para evitar la recepción de archivos excesivamente grandes.

Generación de nombres aleatorios: se implementó random_bytes() para generar nombres nuevos, evitando utilizar directamente los nombres proporcionados por los usuarios.

Comprobación de la subida: se verificó que el archivo recibido corresponda a una subida válida antes de almacenarlo mediante move_uploaded_file().

sudo nano /var/www/html/DVWA/vulnerabilities/upload/source/low.php

- **Código original**

```
<?php
if( isset( $_POST[ 'Upload' ] ) ) {
    // Where are we going to be writing to?
    $target_path  = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
    $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );
    // Can we move the file to the upload folder?
    if( !move_uploaded_file( $_FILES[ 'uploaded' ][ 'tmp_name' ], $target_path ) ) {
        // No
        $html .= '<pre>Your image was not uploaded.</pre>';
    }
    else {
        // Yes!
        $html .= "<pre>{$target_path} succesfully uploaded!</pre>";
    }
}

?>
```

- **Código modificado**

```
<?php
if (isset($_POST['Upload'])) {
  // Directorio de almacenamiento
  $target_dir = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
  // Comprobar que el archivo fue recibido correctamente
  if (
    !isset($_FILES['uploaded']) ||
    $_FILES['uploaded']['error'] !== UPLOAD_ERR_OK
  ) {
    $html .= '<pre>Error al recibir el archivo.</pre>';
  } else {

    $file = $_FILES['uploaded'];
    // Comprobar el tamaño máximo: 5 MB
    if ($file['size'] > 5 * 1024 * 1024) {
      $html .= '<pre>El archivo supera el tamaño permitido.</pre>';
```

```
} else {
    // Extensiones permitidas y sus MIME correspondientes
    $allowed_types = [
        'jpg'  => 'image/jpeg',
        'jpeg' => 'image/jpeg',
        'png'  => 'image/png'
    ];
    // Obtener extensión
    $extension = strtolower(
        pathinfo($file['name'], PATHINFO_EXTENSION)
    );
    // Validar extensión
    if (!array_key_exists($extension, $allowed_types)) {
        $html .= '<pre>Tipo de archivo no permitido.</pre>';
    } else {
        // Detectar MIME real del archivo
        $finfo = finfo_open(FILEINFO_MIME_TYPE);
        $mime = finfo_file($finfo, $file['tmp_name']);
        finfo_close($finfo);
        // Comprobar que sea una imagen válida
        $image_info = getimagesize($file['tmp_name']);
        if (
            $mime !== $allowed_types[$extension] ||
            $image_info === false ||
            $image_info['mime'] !== $allowed_types[$extension]
        ) {
            $html .= '<pre>El contenido del archivo no es válido.</pre>';
        } else {
            // Generar un nombre aleatorio
            $new_name = bin2hex(random_bytes(16)) . '.' . $extension;
            // Construir la ruta final
            $target_path = $target_dir . $new_name;
            // Mover el archivo validado
            if (
                is_uploaded_file($file['tmp_name']) &&
                move_uploaded_file(
                    $file['tmp_name'],
                    $target_path
                )
            ) {
                $html .= '<pre>Imagen subida correctamente.</pre>';
            } else {
```

```
                $html .= '<pre>No se pudo guardar el archivo.</pre>';
            }
        }
    }
  }
}
?>
```

Además en creamos una configuración de apache que rechace archivos ejecutables

sudo nano /etc/apache2/conf-available/dvwa-uploads.conf

```
<Directory "/var/www/html/DVWA/hackable/uploads">
  Options -ExecCGI
  <FilesMatch "\.(?i:php[0-9]?|phtml|phar)$">
    Require all denied
  </FilesMatch>
</Directory>
```

- **Justificación**

  La mitigación implementada busca impedir que la funcionalidad de subida de archivos sea utilizada para introducir código ejecutable en el servidor.

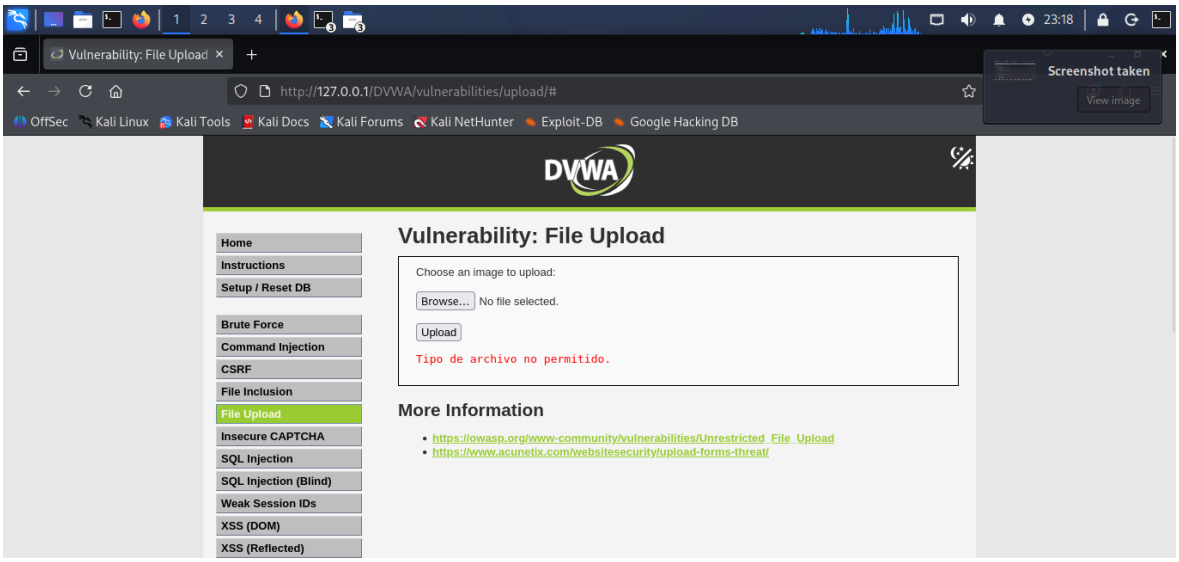
  La lista blanca de extensiones reduce los tipos de archivos que pueden recibirse, mientras que la validación MIME y la comprobación del contenido permiten detectar archivos que intenten hacerse pasar por imágenes legítimas.

  La generación de nombres aleatorios evita depender de los nombres originales enviados por los usuarios y reduce posibles conflictos o manipulaciones de rutas. Por otra parte, el límite de tamaño permite controlar los recursos utilizados durante la recepción de archivos.

  Finalmente, la restricción de acceso configurada en Apache proporciona una capa adicional de protección, evitando que los archivos PHP almacenados en el directorio de subidas puedan ejecutarse o consultarse mediante solicitudes web.

  La solución conserva la funcionalidad legítima de la aplicación, permitiendo subir imágenes JPG y PNG, mientras rechaza archivos que no cumplen las condiciones establecidas.

  La efectividad de la mitigación se comprueba mediante pruebas posteriores a la modificación, verificando el rechazo de archivos PHP y TXT, la aceptación de imágenes válidas y la imposibilidad de ejecutar scripts desde el directorio de almacenamiento.


1 2 3 4
23:18
Vulnerability: File Upload
http://127.0.0.1/DVWA/vulnerabilities/upload/#
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
Screenshot taken
View image
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
Vulnerability: File Upload
Choose an image to upload:
Browse...
No file selected.
Upload
Tipo de archivo no permitido.
More Information
https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
https://www.acunetix.com/websitesecurity/upload-forms-threat/

4.3. Conclusiones generales

El desarrollo de este proyecto permitió analizar la seguridad de una aplicación web mediante la identificación, explotación controlada y mitigación de cuatro vulnerabilidades: SQL Injection, Stored Cross-Site Scripting (XSS), Brute Force y Unrestricted File Upload. A través de las pruebas realizadas en DVWA, fue posible comprender cómo las deficiencias en la implementación de mecanismos de seguridad pueden comprometer la confidencialidad, integridad y disponibilidad de la información, así como afectar el funcionamiento general de una aplicación.

Durante el análisis de SQL Injection se evidenció la importancia de utilizar consultas preparadas para evitar que las entradas proporcionadas por los usuarios alteren la estructura de las consultas a la base de datos. Por otra parte, el estudio de Stored XSS permitió reconocer la necesidad de aplicar mecanismos adecuados de validación y codificación de salida para impedir que el contenido proporcionado por los usuarios sea interpretado como código ejecutable por el navegador.

Asimismo, el análisis de la vulnerabilidad Brute Force permitió comprender la importancia de implementar controles sobre los intentos de autenticación, mecanismos de bloqueo temporal y retardos progresivos que dificulten los intentos automatizados de acceso no autorizado. En cuanto a Unrestricted File Upload, se identificó la necesidad de establecer restricciones sobre los archivos recibidos por una aplicación, mediante listas blancas de extensiones, validación del contenido, límites de tamaño y configuraciones adicionales del servidor web.

La implementación de las mitigaciones permitió aplicar diferentes medidas de seguridad directamente sobre el código fuente y la configuración del entorno. Este proceso también permitió reconocer que una solución no debe limitarse a modificar el comportamiento vulnerable, sino que debe comprobarse mediante pruebas posteriores para determinar si realmente reduce el riesgo sin afectar las funcionalidades legítimas del sistema.

Finalmente, el proyecto permitió fortalecer los conocimientos relacionados con el análisis de seguridad de aplicaciones web, el uso de herramientas de evaluación y la aplicación de buenas prácticas de desarrollo seguro. Se concluye que la seguridad debe considerarse una parte fundamental durante todo el ciclo de desarrollo de software, debido a que la prevención, identificación y corrección temprana de vulnerabilidades contribuyen a construir aplicaciones más confiables y resistentes frente a posibles amenazas.

4.4. Reflexión ética

El desarrollo de este proyecto se realizó bajo los principios de responsabilidad, legalidad y uso ético de las herramientas de hacking, entendiendo que los conocimientos adquiridos en ciberseguridad deben utilizarse para identificar riesgos, fortalecer sistemas y prevenir accesos no autorizados, y no para causar daños o comprometer la información de otras personas.

Todas las actividades de identificación y explotación de vulnerabilidades se llevaron a cabo sobre una instancia local y controlada de Damn Vulnerable Web Application (DVWA), destinada específicamente al aprendizaje y la práctica de técnicas de seguridad informática. De esta manera, las pruebas se mantuvieron dentro de un entorno autorizado, sin intervenir sistemas de terceros ni afectar servicios externos.

El uso de herramientas como Burp Suite y las técnicas de análisis de vulnerabilidades permitió comprender que las capacidades técnicas de un profesional en Ingeniería de Sistemas implican también responsabilidades sobre el manejo de la información, la privacidad de los usuarios y la protección de los recursos tecnológicos.

Asimismo, este ejercicio permitió reflexionar sobre la importancia de comunicar los hallazgos de seguridad de manera responsable, documentar las vulnerabilidades identificadas y proponer soluciones que contribuyan a mejorar la protección de las aplicaciones. La información obtenida durante las pruebas fue utilizada con fines académicos y de aprendizaje, manteniendo el alcance establecido para el proyecto.

Como futuros profesionales de Ingeniería de Sistemas, reconocemos que el hacking ético debe estar fundamentado en la autorización, la transparencia y el respeto por los sistemas y la información. Por esta razón, consideramos que los conocimientos adquiridos deben orientarse a la prevención de amenazas, el fortalecimiento de la seguridad informática y el desarrollo de soluciones tecnológicas confiables.

Referencias

digininja/DVWA: Damn Vulnerable Web Application (DVWA)

GIT del proyecto

https://github.com/Millo674/proyectoFinalHE.git